



Tries & Digital Search Structures

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

Introduction to Tries

A Trie (derived from 'retrieval') is an ordered tree-like data structure used to store a dynamic set of strings, where keys are usually strings. It is also known as a prefix tree.

Core Structure & Design

- **Key Sharing:** Unlike BSTs, nodes do not store their associated keys. Instead, a node's position in the tree defines its key.
- **Character Edges:** Edges outgoing from a node represent characters in the alphabet. All descendants of a node share a common string prefix.
- **End of Word Marker:** A boolean flag (`isEndOfWord`) indicates if the path from root to that node represents a complete word in the set.
- **Root Node:** Always represents the empty string (root of all search prefixes).

Key Complexity & Benefits

- **$O(L)$ Operations:** Lookup, insert, and delete run in $O(L)$ time, where L is the length of the string. Completely independent of table size n !
- **Fast Misses:** Searches terminate immediately when a character edge is missing — much faster than hashing on key mismatches.
- **Prefix Queries:** Can easily retrieve all keys starting with a given prefix, which is impossible with standard hash tables.
- **Space Trade-off:** Tries can use significant memory because of pointer overhead for empty edges at each node.

Standard Tries vs. Binary Tries

Tries can be implemented using different alphabet sizes and structures:

Standard Tries (Multiway Tries)

- **Alphabet size (Σ):** Typically large (e.g. $\Sigma = 26$ for lowercase English, $\Sigma = 256$ for ASCII).
- **Child Representation:** Each node stores an array of size Σ pointing to child nodes: `Node* children[26]`.
- **Pointer overhead:** Wastes substantial memory. If a node only has one child, 25 array slots are empty pointers.
- **Search Speed:** Extremely fast. Direct array access `child = node.children[char]` in $O(1)$ step time.

Binary Tries

- **Alphabet size (Σ):** Binary alphabet $\Sigma = \{0, 1\}$.
- **Bit-level Representation:** Keys are split into their binary bit strings. The edges represent 0 or 1 bits.
- **Child Representation:** Node only has two child pointers: left (for 0) and right (for 1).
- **Space Efficiency:** High. Minimum pointer waste compared to standard multiway tries.
- **Bit-level search:** Searching involves traversing bit by bit. Useful in routing tables where IP addresses are represented as 32-bit strings.

Compressed Tries & Patricia Tries

To solve the space overhead of tries, we compress paths with single-child nodes:

Compressed Tries

- **The Idea:** If a chain of single-child nodes exists (e.g. $p \rightarrow a \rightarrow t \rightarrow h$), collapse them into a single node with edge label 'path'.
- **Structural Benefit:** Reduces the number of internal nodes. Every internal node in a compressed trie now has at least two children.
- **Size Bound:** For a set of n words, the number of leaf nodes is n , and the number of internal nodes is at most $n-1$. Total space is $O(n)$.

Patricia Tries

- **Definition:** PATRICIA stands for: 'Practical Algorithm to Retrieve Information Coded in Alphanumeric'.
- **Compact Binary Trie:** A compressed binary trie. Instead of storing edge characters, nodes store the bit-index to be tested next.
- **Fast Bit Testing:** When searching, we skip bit positions that do not distinguish between keys, jumping directly to the bit position where they diverge.
- **Application:** Highly efficient for prefix search, IP routing lookups, and text processing.

Suffix Trees

A **Suffix Tree** is a compressed trie containing all suffixes of a single string S . It is a powerful index for text searching.

How it is Built

- **Suffix Set:** For a string S of length n , append a termination character '\$'. Generate all n suffixes of S .
- **Example ($S = \text{'BANANA\$'}$):** Suffixes are: 'BANANA\$', 'ANANA\$', 'NANA\$', 'ANA\$', 'NA\$', 'A\$', '\$'.
- **Trie Compression:** Insert all these suffixes into a compressed trie.
- **Linear Construction:** Ukkonen's algorithm builds a suffix tree in $O(n)$ time and space.

String Operations

- **Pattern Matching:** Check if pattern P is a substring of S in $O(|P|)$ time. We just trace P from root.
- **Longest Repeated Substring:** Find the deepest internal node in the tree. Can be solved in $O(n)$ time.
- **Longest Common Substring:** Build a generalized suffix tree for S_1 and S_2 . Deepest node with descendants from both strings is the solution. $O(n_1 + n_2)$ time.
- **Substring Count:** Find the number of occurrences of P in $O(|P|)$ time.

Applications of Digital Search Structures

Tries and suffix trees form the core of modern search, routing, and text analysis:

Autocomplete & Typeahead

Tries store dictionary terms. As the user types, the prefix acts as a path to a node. The sub-tree under that node is traversed to yield suggestions in $O(L)$ time, where L is prefix length.

IP Routing Table Lookups

Patricia tries are used in network routers to store routing tables. The router performs a 'Longest Prefix Match' on the incoming packet's IP address to find the forwarding route in $O(1)$ time.

Spell Checkers & Dictionaries

Tries act as spelling dictionaries. It is extremely fast to detect if a word exists, and to list nearby word candidates using spelling edit distance algorithms on the trie.

Bioinformatics & DNA Indexing

Suffix trees index massive genomes. Researchers search for DNA sub-sequences, repeating genes, and similarities across species in linear time using suffix tree lookups.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management